Code guide: the segmentation pipeline

C. H. Ross. 2026-11-05. Distributed with the package source.

1. Scope

This chapter describes the code that turns a NARWC survey file into segments, detections and a `Distance` flatfile. Each section takes one chunk of code, in the order `segment_survey()` runs it, and states what it does, what it reads and writes, and why.

Covered: `prepare_survey()`, `segment_survey()` and the functions it calls, and `detection_data()`. Not covered: plotting, `diagnose_pipeline()`, `line_effort()`, spatial export. Functions from the `narwcr` package (reading, line identity, effort flags) are described at their interface only.

Line references are to the source on 2026-11-05. Design reasoning is in `docs/02-implementation.md`; defects in the original scripts are in `docs/03-bug-fixes.md`. For the analysis steps from a file to an abundance estimate, see `docs/11-distance-sampling-steps.md`.

Terms:

Term	Meaning
record	One row of the survey file: a position fix, an event, or a sighting.
line occupation	One continuous flight along one survey line. A line flown twice in a day is two occupations. Column `LEGNO3`.
on effort	On a census line in acceptable conditions. Column `OnOff.Effort == 1`.
track	A run of consecutive records in one occupation with the same effort state. Column `new_trackno`, numbered within a `DATE`.
segment	A piece of an on-effort track of about `seg_length` km. The sampling unit of a density surface model. Column `seg_id`.
`chopped`	The record-level table after cutting: on-effort records, each labelled with its segment.
circling	Leaving the census line to circle a sighting. Starts at a break-off record (`LEGSTAGE == 3`) and ends at a resume record (`LEGSTAGE == 4`).
break-off	The `LEGSTAGE == 3` record: the point where the aircraft left the line.

2. Pipeline summary

Step	Function	File	Adds
0	`read_narwc()` (narwcr)	re-exported	standard column names and types
1	`make_leg_id()` (narwcr)	re-exported	`LEGNO2`, `LEGNO3`
1a	`fill_legstage()` (narwcr)	called internally	filled `LEGSTAGE`, `LEGSTAGE_FILLED`
1b	`classify_platform()` (narwcr)	called internally	`PLATFORM_KIND`; rows filtered
1c	`flag_circling()`	`R/circling.R:35`	`CIRCLE`
2	`flag_effort()` (narwcr)	`R/platform.R:165`	`OnOff.Effort`
3	`point_to_point_effort()`	`R/geodist.R:312`	`pt2pt.effort`, `Effort`
3a	`sighting_distances()`	`R/distances.R:201`	`distance`, `distbegin`, `distend`, `side`, `distance_source`
4	`split_tracks()`	`R/tracks.R:56`	`new_trackno`
5	`track_effort()`	`R/tracks.R:125`	one row per track
6	`plan_segments()`	`R/segments.R:87`	one row per planned segment
7	`cut_segments()`	`R/segments.R:219`	`seg_no`, `seg_id`, `seg_eff`, `events`, `case`
7a	`attach_circling_sightings()`	`R/circling.R:233`	circling sightings added to `chopped`
8	`segment_midpoints()`	`R/midpoints.R:115`	`mid_lat`, `mid_lon`
9	`segment_sightings()`	`R/sightings.R:97`	sightings, conditions and detections tables
10	`assemble_segments()`	`R/segment-survey.R:302`	the `segments` table
11	`detection_data()`	`R/detection-data.R:135`	the `Distance` flatfile

There are two entry points. `prepare_survey()` runs steps 1, 1a, 1b and 2. `segment_survey()` runs steps 1, 1a, 1c and 2 onwards. It skips step 1, 1a or 2 when the input already has `LEGNO3`, `LEGSTAGE_FILLED` or `OnOff.Effort`. It does not filter by platform (step 1b). The recommended order is:

```
dat  <- read_narwc(path)
dat  <- prepare_survey(dat, platform = "aerial")
segs <- segment_survey(dat, seg_length = 5, seed = 1)
flat <- detection_data(segs, area = 5811)
```

3. `prepare_survey()` — `R/prepare.R:101`

3.1 Platform defaults

```
platform <- resolve_platform(platform, allow_all = TRUE)
if (platform != "all") {
  defaults <- platform_defaults(platform)
  keep_names <- setdiff(names(defaults$effort_args), names(effort_args))
  effort_args <- c(effort_args, defaults$effort_args[keep_names])
}
```

`resolve_platform()` (`R/platform.R:108`) treats `"ship"` as `"vessel"`. `platform_defaults()` (`R/platform.R:85`) gives each platform's effort arguments and distance sources:

Platform	`effort_args`	`distance_sources`
aerial	none (narwcr defaults)	`angle`, `exact`, `strip`
vessel	`max_alt_m = NULL`	`exact`

Arguments passed by the caller override the defaults, name by name. `modifyList()` is not used, because it deletes elements set to `NULL`, and a vessel survey needs `max_alt_m = NULL`.

3.2 Line occupations and line state

```
dat <- narwcr::make_leg_id(dat, quiet = quiet)
if (fill_legstage && "LEGSTAGE" %in% names(dat)) {
  dat <- narwcr::fill_legstage(dat, quiet = quiet)
}
```

`make_leg_id()` sorts records by `DATE`, `FILEID` and `EVENTNO` and assigns `LEGNO3`. An occupation starts at a begin-line record (`LEGSTAGE == 1`) or a change of `LEGNO`, and ends at an end-line record (`LEGSTAGE == 5`). Records outside any line (transit, ferry between lines) get `LEGNO3 = NA`.

Survey programmes often record `LEGSTAGE` only when it changes. `fill_legstage()` writes `2` (line continuing) on records with no `LEGSTAGE` that lie between a begin or resume record and the next break-off or end. It writes no other value. This matters because distances are measured only on records with `LEGSTAGE == 2` (Section 4.5).

3.3 Platform filter

```
dat$PLATFORM_KIND <- narwcr::classify_platform(dat)
keep <- !is.na(dat$PLATFORM_KIND) & dat$PLATFORM_KIND == defaults$kind
dat <- dat[keep, , drop = FALSE]
```

Each stretch of records is labelled aerial, vessel or stationary from its median speed. Records of other platforms are removed. The filter runs after `make_leg_id()`. If it ran first, two occupations of the same line could become adjacent and merge, and the ferry between them would count as effort.

Without `LATITUDE`, `LONGITUDE` and `TIME`, no label can be assigned. A warning is given and all records are kept.

3.4 Corrections and effort

```
if (!is.null(correct)) dat <- correct(dat)
if (!"OnOff.Effort" %in% names(dat)) {
  dat <- flag_effort_for_platform(dat, effort_args)
}
```

`correct` is an optional user function. It runs after the platform filter and before the effort flags, so a corrected column (for example `ALT` recorded in feet) is what the effort criteria see. It must return the same number of rows.

`flag_effort_for_platform()` (`R/platform.R:165`) calls `narwcr::flag_effort()`. A record is on effort when `LEGTYPE == 2`, `BEAUFORT <= 3`, `ALT < 366` m, and visibility is at least 2 nmi. A missing value fails its criterion. The wrapper first checks that the installed narwcr accepts `NULL` to drop a criterion, and stops with an explanation if not.

4. `segment_survey()` — `R/segment-survey.R:135`

The function body is a fixed sequence of steps. Each subsection is one step.

4.1 Arguments and settings

```
reject_removed_inherit(circling_distance, "circling_distance")
circling_distance <- match.arg(circling_distance)
defaults <- platform_defaults(platform)
if (is.null(distance_sources)) {
  distance_sources <- defaults$distance_sources
} else {
  check_distance_sources(distance_sources, platform)
}
if ("circling" %in% distance_sources) {
  rlang::warn(...)
  distance_sources <- setdiff(distance_sources, "circling")
}
```

`reject_removed_inherit()` (`R/circling.R:579`) stops with an explanation when the removed option `"inherit"` is passed. It runs before `match.arg()`, which would otherwise give a generic error. `check_distance_sources()` (`R/platform.R:128`) stops when an aircraft-only source (`angle`, `strip`) is requested for a vessel.

The `"circling"` distance source is dropped with a warning. Circling records are off effort, so they are not in `chopped`; their distance is set when they are attached (Section 4.8).

All resolved arguments are stored in `settings` and returned with the result.

4.2 Step 1: line occupations, line state, platform check, circling flag

```
if (!"LEGNO3" %in% names(dat)) dat <- make_leg_id(dat)
if ("LEGSTAGE" %in% names(dat) && !"LEGSTAGE_FILLED" %in% names(dat)) {
  dat <- narwcr::fill_legstage(dat)
}
if (all(c("LATITUDE", "LONGITUDE", "TIME") %in% names(dat))) {
  kind  <- narwcr::classify_platform(dat)
  other <- !is.na(kind) & kind != defaults$kind
  if (any(other)) rlang::warn(...)
}
if (!"CIRCLE" %in% names(dat)) dat <- flag_circling(dat)
```

Line occupations and line state are built as in Section 3.2, unless `prepare_survey()` has already done so.

The platform is checked but not filtered. If any records are moving at the wrong speed for the platform, a warning gives their number and the `prepare_survey()` call that would remove them. `segment_survey()` never removes rows.

`flag_circling()` (`R/circling.R:35`) sets `CIRCLE = 1` on a record when `LEGTYPE == 4`, or when it lies between a break-off and the next resume in the same occupation. The break-off and resume records themselves get `0`.

4.3 Step 2: effort

```
if (!"OnOff.Effort" %in% names(dat)) {
  dat <- flag_effort_for_platform(dat, effort_args)
}
```

As in Section 3.4. Off-effort records are kept; the circling step needs them.

4.4 Step 3: along-track distance — `R/geodist.R:312`

```
out <- dplyr::group_by(dat, DATE, FILEID, LEGNO3)
.step = gc_distance(LATITUDE, LONGITUDE, lead(LATITUDE), lead(LONGITUDE))
pt2pt.effort = ifelse(!is.na(.next_on) & OnOff.Effort == 1 & .next_on == 1,
                      .step, 0)
```

Within each occupation, the great-circle distance (km) from each record to the next is computed and assigned to the first record of the pair. It counts as effort only when both records are on effort. The last record of an occupation has no next record and gets `0`, so no distance is counted across a gap between occupations.

`dist_method` selects the distance formula; haversine is the default. `source = "recorded"` uses the logged `TRKDIST` instead, and falls back to the great-circle distance, with a warning, where `TRKDIST` is missing.

4.5 Step 3a: perpendicular distances — `R/distances.R:201`

```
eligible <- on_effort_census_rows(dat)
for (src in sources) {
  open <- is.na(source)
  got  <- distance_from_source(dat, src, ...)
  take <- open & eligible & (!is.na(got$distance) | !is.na(got$distbegin))
  distance[take] <- got$distance[take]; source[take] <- src
}
```

Distances are computed before cutting, so each sighting carries its distance into its segment. A record is eligible only if it is on effort, on a census line, and has `LEGSTAGE == 2` (handbook 8.A.31).

`sources` is an order of preference. Each source fills only the records that no earlier source filled. `distance_from_source()` (`R/distances.R:283`) computes one source:

Source	Computation	Result
`angle`	`ALT / tan(angle)` from `ANGLEL` or `ANGLER`	point distance. If both angles are recorded, `side = "both"` and no distance.
`exact`	`S_LAT`/`S_LONG` projected onto the track (`exact_distance()`, `R/exact.R:318`)	point distance
`strip`	`STRIP` code looked up in the CETAP or NLPSC code book, chosen by date	interval, `distbegin` to `distend`
`circling`	`S_LAT`/`S_LONG` projected onto the line at the break-off (`circling_distance()`, `R/exact.R:439`)	point distance; not restricted to eligible records

When a source gives only the side of the track, the side is recorded and the record is left open for a later source.

4.6 Steps 4–5: tracks — `R/tracks.R`

```
track_index <- function(leg, on_effort) {
  new_leg  <- c(FALSE, key[-1] != key[-n])
  isolated <- on == 0L & prev_on == 1L & next_on == 1L
  state[isolated] <- 1L
  changed  <- c(FALSE, state[-1] != state[-n])
  as.character(cumsum(new_leg | changed) + 1L)
}
```

`split_tracks()` numbers tracks within each `DATE`. A new track starts at each change of `LEGNO3` and each change of effort state. A single off-effort record between two on-effort records does not start a new track, so one bad position fix does not split a track. That record still has `pt2pt.effort = 0`.

`track_effort()` sums `pt2pt.effort` per track and drops tracks of `min_track_km` (default 1 km) or less. Off-effort tracks have zero effort, so they are dropped here.

4.7 Steps 6–7: planning and cutting segments — `R/segments.R`

Both steps use random numbers. They run inside one `with_optional_seed()` block, so one `seed` reproduces the whole segmentation.

Planning. `plan_one_track()` (`R/segments.R:119`) sets target lengths for one track of length `total`:

```
n_whole  <- floor(total / seg_length)
leftover <- total - n_whole * seg_length
if (leftover >= seg_tol || n_whole == 0) { n_seg <- n_whole + 1L; extra <- leftover }
```

```
else                                    { n_seg <- n_whole;      extra <- seg_length + leftover }
tgt <- rep(seg_length, n_seg)
tgt[sample.int(n_seg, 1L)] <- extra
```

If the remainder is at least `seg_tol` (`seg_tol_frac * seg_length`; default half a segment), it becomes its own segment. Otherwise one segment is lengthened to include it. The segment that gets the odd length is chosen at random. A track shorter than `seg_length` is one segment. This is the method of Becker et al. (2010), kept unchanged (`docs/02-implementation.md`).

Cutting. `cut_one_track()` (`R/segments.R:248`) assigns the track's records to the planned segments in order:

```
cut_at <- cumsum(plan_i$tgtdist)
tgt <- max(cut_at[i] - covered, 0)
if (i == n_seg)            take <- length(idx)                # case5
else if (eff[1] >= tgt)    take <- 1L                         # case2
else if (sum(eff) <= tgt)  take <- length(idx)                # case4
else { k <- which(cumsum(eff) >= tgt)[1]
       if (runif(1) >= 0.5 && k > 1L) k <- k - 1L; take <- k }  # case3
```

A segment can only end at a record. In the usual case (`case3`), the first record that reaches the target is found, and a random draw decides whether the segment ends at that record (slightly long) or the one before (slightly short). Targets are measured from the start of the track, so these small errors do not add up along the track. The last segment takes all remaining records, so no effort is lost.

Each segment gets `seg_id` (`DATE_track_segno`), `seg_eff` (sum of `pt2pt.effort`, km), `events` (first and last `EVENTNO`) and `case`. Segments of `min_segment_km` or less are dropped.

4.8 Step 7a: circling sightings — `R/circling.R:233`

Sightings made while circling are off effort and are not in `chopped`. `attach_circling_sightings()` adds them back.

```
leaked <- CIRCLE == 1 & SPECCODE != ""          # remove any already in chopped
cand   <- dat[CIRCLE == 1 & SPECCODE != "", ]   # candidates
ref_ev <- latest LEGSTAGE == 3 event at or before each candidate
anchor <- latest chopped record at or before ref_ev, same day
target <- segment of anchor
```

1. **Clean-up.** Circling sightings already in `chopped` are removed. This happens when a circle is logged as a single record, which does not split the track (Section 4.6). Without this step such a sighting was counted twice.
2. **Candidates.** Circling records with a species code.
3. **Break-off.** For each candidate, the break-off is the latest `LEGSTAGE == 3` record at or before it in the same occupation. Without `LEGSTAGE`, the candidate's own event is used instead.
4. **Anchor and target segment.** The anchor is the last on-effort record at or before the break-off. Normally this is the break-off record itself. The candidate is attached to the anchor's segment. Distances below are measured from the anchor's position.
5. **Group.** The on-effort sighting the circled animals belong to: the most recent sighting of the same species in the same occupation that is
   - at or before the break-off,
   - after the previous resume (`LEGSTAGE == 4`), and
   - within `max_gap_km` of the anchor (default `seg_length`).

   It may be in an earlier segment than the target.

6. **Rule.** Under `mode = "same_species"` (default; the CETAP rule, handbook 4.2), a candidate with no group is not attached. `"all"` attaches every candidate with a target segment. `"none"` attaches none.
7. **Effort.** Attached records get `pt2pt.effort = 0` and `case = "circling"`. Segment lengths do not change.
8. **Distance.** `break_off_distance` is always computed: metres from the anchor to the logged position (the aircraft's position during the circle). Then:

`circling_distance`	Effect
`"with_group"` (default)	The circled animals (`NUMBER`) are added to the group's `NUMBER`. The circling record's own `NUMBER` is set to 0, the number moved is kept in `circling_merged`, and `circling_counted = FALSE` so it is not counted again. A candidate with no group (only possible under `"all"`) remains a detection with no distance.
`"break_off"`	The candidate is a separate detection. If it has `S_LAT`/`S_LONG`, its distance is that position projected onto the line at the break-off, with `distance_source = "circling"`. Otherwise its distance is `break_off_distance`, with `distance_source = "break_off"`; this is a straight-line distance, not a perpendicular one. Converted to `distance_units`.

All search steps use one `findInterval()` call per day or per occupation, so run time grows linearly with the number of records.

4.9 Steps 8–9: segment summaries

Midpoints (`R/midpoints.R:115`). The midpoint is the position half-way along the segment's effort. It is interpolated on the great circle between the record where the half-way point falls and the next record in the track. The next record is taken from the track, not the segment, because a segment's last record measures its effort to the next segment's first record. A segment of zero length uses its first position.

Sightings, conditions and detections (`R/sightings.R:97`).

```
sight <- filter(chopped, SPECCODE != "")
sight <- filter(sight, is.na(circling_counted) | circling_counted)
sight <- filter(sight, is.na(LEGSTAGE) | !LEGSTAGE %in% c(6, 7))
sight <- filter(sight, is.na(IDREL)    | IDREL %in% c(2, 3))
```

Conditions are the mean and the effort-weighted mean `BEAUFORT` per segment. Sightings are filtered: circling records already counted with their group are removed, as are `LEGSTAGE` 6 and 7, and identifications not rated probable or definite (`IDREL` 2 or 3). `species` restricts further. Two tables result:

- `sightings`: number of sightings and animals per segment and species.
- `detections`: one row per sighting, with `distance`, `distbegin`, `distend`, `side`, `distance_source`, `size`, `circling` and `break_off_distance`.

4.10 Step 10: segments table — `R/segment-survey.R:302`

One row per segment: date, line identifiers, `seg_no`, `seg_eff`, start time, `events` and `case`, joined to the midpoints and conditions.

`segment_survey()` returns a `distsamp_segments` object containing `segments`, `sightings`, `detections`, `tracks`, `points` (`chopped`), the call, and `settings`.

5. `detection_data()` — `R/detection-data.R:135`

```r
base <- tibble(Region.Label, Area = area, Sample.Label = seg_id, Effort = seg_eff)
det  <- det[!(circling == 1), ]            # unless include_circling = TRUE
det  <- det[!is.infinite(distend), ]       # open top STRIP bin
det  <- det[!(distance > truncation), ]    # if truncation is given
```

Every segment is a sampling unit, including segments with no detections. `area` is required and has no default, because it scales abundance directly. The following are removed:

- circling detections (unless `include_circling = TRUE`);
- detections in the open top `STRIP` bin, which has no upper limit and cannot be fitted;
- detections beyond `truncation`. Their segments keep their effort.

A table with both point and interval distances is an error by default, because `ds()` cannot fit both at once. The output has a `distance` column for point distances, or `distbegin`/`distend` for intervals, not both: `Distance::ds()` treats any table with the interval columns as interval data. A summary of what was removed is printed, with a reminder that `g(0) = 1` is assumed.

6. Units

Quantity	Unit
`pt2pt.effort`, `seg_eff`, `track_effort`, `Effort`	km
`distance`, `distbegin`, `distend`	`distance_units`: m (default) or km
`break_off_distance`	m
`Area`	as supplied by the caller

With effort in km and distances in m, `Distance::ds()` needs `convert_units = 0.001`. Without it, density is wrong by a factor of 1000. Alternatively use `distance_units = "km"`.

7. Order of steps

Each rule below is enforced by the code.

Rule	Enforced in	If broken
Line occupations before the platform filter	`prepare_survey()`	Two occupations of one line merge; the ferry between them counts as effort.
Line state filled before distances	`prepare_survey()`; `segment_survey()` step 1	Mid-line sightings with no `LEGSTAGE` get no distance (85% of sightings in one real archive).
Corrections before effort flags	`prepare_survey(correct = )`	A corrected `ALT` is not used by the altitude criterion.
Circling flag before effort and cutting	`segment_survey()` step 1	Circling records cannot be identified after cutting.
Distances before tracks and cutting	`segment_survey()` step 3a	Distances would have to be re-joined to segments.
Along-track distance within occupations	`point_to_point_effort(by = )`	Distance is counted across gaps between occupations.
One seed for planning and cutting	`with_optional_seed()`	Segmentation cannot be reproduced from `seed`.
Circling sightings attached after cutting, with zero effort, only once	`attach_circling_sightings()`	Segment lengths change, or circled animals are lost or counted twice.
Circling distance set only at attachment	`attach_circling_sightings()`	A distance computed earlier is overwritten.

8. Defects corrected on 2026-11-05

Defect	Correction
`man/platform_defaults.Rd` was malformed. An indented paragraph was read as a code block. Two `R CMD check` warnings.	Paragraph re-indented; Rd regenerated.
`segment_survey(distance_sources = "circling")` had no effect and gave no message.	Warns and drops the source.
`segment_survey(circling_distance = "inherit")` gave a generic error, not the explanation. The test did not detect this.	`reject_removed_inherit()` runs first; the test checks for the explanation.
Help pages described the removed `"inherit"` option as current.	Rewritten.
`flag_circling()` warned on a table with no `DATE` column.	Fixed.
`diagnose_pipeline()` reported "0 survey day(s)" with no `DATE` column.	Reports "an unknown number of".
A test let an expected warning through.	Wrapped in `expect_warning()`.
`circling_distance = "break_off"` used the straight-line distance from the break-off even when the sighting position (`S_LAT`/`S_LONG`) was recorded.	Uses the perpendicular distance when the position exists. Fixture: 523.9 m becomes 250.0 m, the position it was built with.
`circling_distance = "break_off"` wrote metres into `distance` when `distance_units = "km"`.	Converted to `distance_units`.
`segment_survey()` did not fill `LEGSTAGE` on unprepared data, so mid-line sightings without it got no distance.	Filled unless `prepare_survey()` has done it.
`detection_data()` always wrote `distbegin`/`distend`. `Distance::ds()` then treated point data as interval data and failed.	Only the columns for the type of distance present are written.
Circled animals were lost when a segment boundary fell between the sighting and the break-off.	The group is searched for in the whole occupation, after the previous resume, within `max_gap_km`.
A circling sighting was attached to the segment that ended before the circle, not the break-off's segment. For a circle logged as one record, this was the previous segment, and `break_off_distance` was measured from its far end (5,556 m instead of 0 m).	Attached to the segment containing the break-off; distances measured from the break-off.
A circling sighting logged as one record was counted twice, and was counted even under `circling = "none"`.	Removed from `chopped` before attachment.
The vignette `from-segments-to-density.Rmd` (code not run on build) omitted `convert_units` in `ds()` and `convert.units` in `dsm()`, and used an undefined `df_data`.	Corrected.

9. Limitations

- `segment_survey()` does not filter by platform. Unprepared data is segmented under one platform's criteria, with a warning.
- Under `circling = "all"` with `"with_group"`, a circling sighting of a species not seen from the line has no group and no distance. It is counted in `sightings` but cannot be used in a detection function.
- `max_gap_km` defaults to `seg_length`. This is a judgement, not a protocol value. A group seen further from the break-off is not joined.
- Under `"break_off"`, the circling position is recorded minutes after the detection. The distance is therefore not measured at the time of detection. Circling detections are excluded from `detection_data()` by default.
- The narwcr functions are described at their interface only.
- All results in Section 10 are from the synthetic fixture. No real data was used.

10. Verification performed

Check	Result
`R CMD check --no-manual`, before corrections	0 errors, 2 warnings, 0 notes. 756 tests pass; 3 test warnings.
`R CMD check --no-manual`, after corrections	0 errors, 0 warnings, 0 notes. 794 tests pass; 0 skipped; 0 test warnings.
Effort on the fixture (`seg_length = 5`, `seed = 1`)	113 records, 2 days, 6 occupations, 101 on effort, 5 circling. 92.23 km of effort; 20 segments totalling 92.23 km. 8 detections, all from `angle`.
`"break_off"` on the fixture	Circling sighting at 250.0 m, right side, `distance_source = "circling"`. With `S_LAT` removed, falls back to the straight-line distance.
Group search, test scenario, `max_gap_km = 5`	Groups up to 4.5 km before the break-off are joined; groups 5.6 km or more before are not, whether or not a segment boundary lies between. A group 22 km back is not joined. A group circled before the last resume gets no further animals.
Circle logged as one record	Attached to the break-off's segment; `break_off_distance` 0 m (was 5,556 m). Right whales counted: 3 (was 4) under `"same_species"`; 2 (was 3) under `"none"`.

Check	Result
Full analysis on the fixture	`ds()` and `dsm()` run on the output; see `docs/11-distance-sampling-steps.md`, Section 14.

11. Planned work

1. Check `max_gap_km` against real data, using the distances between sightings and their break-offs.
2. Run the pipeline on a real extract (`docs/08-onboarding-a-real-extract.md`).

12. References

Becker, E.A., Forney, K.A., Ferguson, M.C., Foley, D.G., Smith, R.C., Barlow, J. and Redfern, J.V. (2010) Comparing California Current cetacean-habitat models developed using in situ and remotely sensed sea surface temperature data. *Marine Ecology Progress Series* 413:163–183.

CETAP (1982) *A Characterization of Marine Mammals and Turtles in the Mid- and North-Atlantic Areas of the U.S. Outer Continental Shelf, Final Report.* University of Rhode Island.

Kenney, R.D. (2023) *The North Atlantic Right Whale Consortium Database: A Guide for Users and Contributors, Version 8.* NARWC Reference Document 2023-01. Sections 4.2, 8.A.1, 8.A.2, 8.A.19–8.A.21, 8.A.31, 8.A.33–8.A.34.

Kenney, R.D. and Winn, H.E. (1986) Cetacean high-use habitats of the northeast United States continental shelf. *Fishery Bulletin* 84(2):345–357.